Mini-RAG & Local AI with Ollama

Merouane Mouadili
DSAI2S Research Team, C3S Laboratory, FST of Tangier Abdelmalek Essaâdi University Tetouan, Morocco

Report — Answers to the questions (Part 18)

Q1. What is Local AI?

Local AI means running an artificial intelligence model directly on your own computer instead of on a remote server. The model is downloaded once and then works without sending any data outside the machine. In this lab, the models ran on my laptop with Ollama.

Q2. Difference between a Cloud LLM and a Local LLM?

A cloud LLM runs on the provider's servers. The user sends the prompt over the Internet and pays per use. It is usually fast and very powerful, but the data leaves the computer.

A local LLM runs on the user's own hardware. It works offline, keeps data private and has no usage cost, but it is limited by the computer's memory and speed. It is also generally less capable than the largest cloud models.

Q3. What role does Ollama play?

Ollama is the tool that makes local models easy to use. It downloads and manages models, loads them into memory, and runs them on the CPU or GPU. It also exposes a local HTTP server on port 11434, so the same model can be used from the terminal, from Python or from a web application.

Q4. What is model quantization?

Quantization reduces the precision of the numbers that store the model's weights, for example from 16 bits to about 4 bits. This makes the model much smaller and faster, with only a small loss of quality. The qwen3.5 model used in this lab is quantized in Q4_K_M, which is why a 9-billion-parameter model fits in 6.6 GB.

Q5. Why does model size affect memory consumption?

To generate an answer, all the model's weights must be loaded into RAM or GPU memory. More parameters means more numbers to store, so a larger model needs more memory. For example, qwen3.5 (9B) used 6.19 GB in memory, while gemma3 (4B) used only 2.89 GB.

Q6. Why can a smaller model sometimes be preferable?

A smaller model needs less memory and generates text faster, so it works better on a normal computer without a GPU. For simple tasks, its answers are often good enough. In my tests, gemma3 was almost twice as fast as qwen3.5 and still produced correct code.

Q7. Advantages of local inference?

The data stays on the machine, which protects privacy. The model works without an Internet connection. There is no cost per request, so we can experiment freely. We also have full control over the model version and its settings.

Q8. Disadvantages of local inference?

Local models are limited by the hardware. On a CPU, they can be very slow: in my tests, generation reached only 3.7 to 6.8 tokens per second. Large models need a lot of RAM, and my computer reached more than 90% memory use. Local models are also usually less accurate than the best cloud models, and the user must install and maintain everything.

Q9. Explain the architecture of your Python application.

The user writes a question in a web page. The browser sends it to a Flask server written in Python (app.py). The server first uses a small retriever (rag.py) to find the most relevant course documents. It then builds a message list with a system instruction, the recent conversation history and the retrieved context, and sends it to Ollama through its REST API.

Ollama runs the local model and returns the answer. Flask sends the answer back to the page, together with the sources used and the response time. When the user asks for a structured answer, a JSON schema forces the model to return a fixed format.

Q10. Explain the role of the REST API.

The REST API is the way programs communicate with Ollama. Each action is an HTTP request to a specific address, such as /api/generate or /api/chat to get an answer, /api/tags to list models, or /api/ps to see loaded models. Data is exchanged in JSON.

Because it uses standard HTTP, any language or tool can use the model, not only Python. Our Flask application works the same way, with its own /ask endpoint used by the web page.

Q11. What is RAG?

RAG means Retrieval-Augmented Generation. Before asking the model, the program searches a set of documents for the parts related to the question and adds them to the prompt. The model then answers from these documents instead of relying only on what it learned during training.

This allows the model to use private or recent information, and to say "Information not available" when the answer is not in the documents.

Q12. Difference between an LLM and a multimodal LLM?

A classic LLM only understands and produces text. A multimodal LLM can also take other inputs, such as images. It uses a vision encoder that converts the image into data the language model can process. In this lab, gemma3 described a photo of a kitten and even read the small text written on it.

Q13. Compare two local models experimentally.

I compared qwen3.5 (9B parameters) and gemma3 (4B parameters) with the same prompt, a 500-word explanation of reinforcement learning, on a computer without a GPU.

Model	Disk size (GB)	Memory (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s
qwen3.5	6.55	6.19	15.2	222.2	758	3.73
gemma3	3.35	2.89	39.8	167.3	821	6.76

Table 1 — Benchmark results measured in Part 16 (CPU only, VRAM = 0 GB)

Gemma3 used half the memory and was about 1.8 times faster, even though it wrote a longer answer. Its load time was longer only because qwen3.5 was still in memory and RAM use reached 96.8%.

In the coding test of Part 6, both models wrote a correct prime-number function. The qwen3.5 code was slightly more optimized, but both made small mistakes in their explanations.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

I would choose gemma3. It needs about 2.9 GB of memory instead of 6.2 GB, it generates text almost twice as fast, and its answers were good enough for our tasks. On my computer, qwen3.5 pushed memory use close to saturation and made everything slower. A larger model is only useful if the computer has enough memory to run it comfortably.

Q15. What security risks exist when exposing a local LLM server to a network?

By default, Ollama only listens on localhost, so only the local computer can use it. If we set OLLAMA_HOST=0.0.0.0, the server becomes reachable from the whole network, and Ollama has no password or authentication by default. Anyone who can reach it could use our computer's power for their own requests, slow down or overload the machine, list our models, download new ones or delete existing ones. Requests are also sent in plain HTTP, so prompts and answers could be read by someone on the same network.

There is also a risk of prompt injection through documents. If a document used by the RAG contains hidden instructions, the model may follow them instead of the user's question. Another risk is leaving Flask in debug mode, since its debugger can allow code execution if the page is reachable from outside.

To reduce these risks, the server should stay on localhost when possible, as our application does with 127.0.0.1. If network access is needed, a firewall should limit who can connect. A reverse proxy such as Nginx can add a password or token and encrypt traffic with HTTPS (TLS). Finally, only trusted documents should be given to the RAG.